SOFTWARE TESTING AND QUALITY ASSURANCE

AlumniConnect

Test Plan, Executed Test Cases, Results and Quality Summary

A repository-grounded testing report for the AlumniConnect alumni networking and mentorship platform.

Report type	STQA Test Execution Report
Execution date	28 October 2026
Project version	Not identified in the executed test scripts
Execution scope	Five existing, local, non-destructive Node.js test scripts
Test evidence	Console results from the scripts listed in Section 3

This report records only the test scenarios found in the repository and actually executed for this run. It does not claim unrun API, database, SMTP, browser, or manual test coverage.

1. Executive Summary

Five existing test scripts were run directly with Node.js. The selected scripts exercise typography and layout source checks, demo-account configuration, password and URL helper behavior, alumni data normalization and privacy sanitization, and duplicate-registration scenarios.

32	28	4	87.5%
Existing cases recorded	Passed	Failed	Pass rate (28 / 32)

Existing test script	Cases	Passed	Failed	Result
`backend/scripts/testFontSelectorAndTypography.js`	8	5	3	**FAIL**
`backend/scripts/testSidebarControlsAndResponsiveness.js`	8	7	1	**FAIL**
`backend/scripts/testAuthAndDemoAccounts.js`	8	8	0	**PASS**
`backend/scripts/test2022ImportAndPrivacy.js`	3	3	0	**PASS**
`backend/scripts/testDuplicateRegistrationFlow.js`	5	5	0	**PASS**
Total	**32**	**28**	**4**	**87.5%**

> **Attention:** The four failures are all in typography assertions. They expose differences between test expectations and the current font configuration; see Section 5. No source-code fixes were made as part of this report generation.

2. Project and Test Scope

AlumniConnect is a web platform connecting students, alumni, teachers and administrators. The selected repository tests cover only the behaviors explicitly asserted by the five executed scripts.

In scope for this execution

- Font/theme wiring and typography configuration through source assertions.
- Sidebar and mobile-drawer structure, keyboard affordances, and responsive breakpoint class presence through source assertions.
- Configured demo-account roles, bcrypt behavior, institutional email examples, LinkedIn URL normalization, and profile privacy sanitization.
- 2022 alumni dataset size and selected data normalization/privacy behavior.
- Duplicate-registration normalization and response/data-preservation scenarios, including the script's explicitly simulated cases.

Not covered by this execution

- Live API workflows, persisted database behavior, database migration/seeding, or production data.

- Real browser rendering at viewport sizes; the responsive checks inspect source/class strings rather than opening a browser.
- Actual SMTP connection or email delivery, load/performance testing, accessibility automation, and manual exploratory testing.
- Any test script not listed in Section 3. No new test cases were invented or executed.

> The backend package defines a smoke-test command that reseeds a database and can fall back to the configured MongoDB URI. It was intentionally not run to avoid changing local or live data. SMTP and database integration scripts were also excluded from this non-destructive run.

3. Test Approach and Environment

Item	Execution detail
Approach	Execute existing Node.js scripts directly; record each printed PASS/FAIL case and preserve the script's actual scope.
Runtime	Node.js in the repository's existing workspace environment. Runtime version was not captured in the execution output.
Database/network	No live database, SMTP service, or external test endpoint was intentionally used by the five selected scripts.
Test data	Script fixtures and repository data, including mock profile/viewer objects and a 48-record import dataset assertion.
Execution commands	`node backend/scripts/<script-name>.js` for each script listed below.

1. `testFontSelectorAndTypography.js`
2. `testSidebarControlsAndResponsiveness.js`
3. `testAuthAndDemoAccounts.js`
4. `test2022ImportAndPrivacy.js`
5. `testDuplicateRegistrationFlow.js`

Counts are derived from the explicit assertions/test blocks reported by the existing scripts. A PASS applies only to the stated assertion; it is not evidence that an entire feature or application flow is production-ready.

4. Detailed Test Cases and Execution Results

4.1 Font selector and typography — `testFontSelectorAndTypography.js`

ID	Test / expected result	Observed result	Status
TC-TYP-01	Configured font choices include the expected Arial, Helvetica, and Times New Roman definitions.	Helvetica definition was not found in `theme.js`.	FAIL
TC-TYP-02	Font application uses `--app-font-family`, persists the choice, and emits the font-change event.	All checked source markers were present.	PASS
TC-TYP-03	Pre-hydration font map contains the exact expected Times New Roman stack.	The exact expected stack was not found; the current map includes a Georgia fallback.	FAIL
TC-TYP-04	Global CSS binds page typography and form controls to the active font variable.	All checked CSS markers were present.	PASS
TC-TYP-05	Tailwind sans and display families bind to the active font variable.	Both configured bindings were found.	PASS
TC-TYP-06	Application startup applies the persisted/initial font and theme.	Both startup calls were found in `main.jsx`.	PASS
TC-TYP-07	Toolbar synchronizes its state through font/theme/storage events and exposes an accessible label.	All checked toolbar markers were present.	PASS
TC-TYP-08	No disallowed external web-font names occur in the inspected typography sources.	Test detected Roboto in the inspected sources and rejected it under its no-Roboto rule.	FAIL

4.2 Sidebar, controls and responsive source checks — `testSidebarControlsAndResponsiveness.js`

ID	Test / expected result	Observed result	Status
TC-UI-01	Desktop sidebar contains toolbar in the intended position above the profile card.	Expected sidebar, navigation, divider and ordering markers were present.	PASS
TC-UI-02	Font selector and theme toggle share a correctly sized horizontal toolbar row.	Expected flex and sizing class strings were present.	PASS
TC-UI-03	Font selector includes type icon, active label and dropdown indicator.	Expected component markers were present.	PASS
TC-UI-04	Theme toggle shows the appropriate icon and accessible action label.	Moon/Sun and label expressions were found.	PASS
TC-UI-05	Font choices include Arial, Helvetica and Times New Roman stacks.	Helvetica stack was not present in the theme source.	FAIL
TC-UI-06	Dropdown source supports Escape, arrow-key navigation and listbox semantics.	Expected keyboard and ARIA markers were present.	PASS

ID	Test / expected result	Observed result	Status
TC-UI-07	Mobile drawer includes the toolbar and profile section.	Expected drawer and toolbar markers were present.	**PASS**
TC-UI-08	Source contains layout class patterns for 320, 375, 768, 1024, 1280 and 1440 px scenarios.	Expected mobile/desktop class markers were found. This is a source assertion, not browser rendering.	**PASS**

4.3 Demo accounts, password, email, URL and privacy — `testAuthAndDemoAccounts.js`

ID	Test / expected result	Observed result	Status
TC-AUTH-01	Configured administrator demo account has the expected institutional role and active/verified properties.	Administrator configuration assertions passed.	**PASS**
TC-AUTH-02	Configured teacher demo account has the expected institutional role and active/verified properties.	Teacher configuration assertions passed.	**PASS**
TC-AUTH-03	Configured student demo account has the expected institutional role and active/verified properties.	Student configuration assertions passed.	**PASS**
TC-AUTH-04	Configured alumni demo account has the expected institutional role and active/verified properties.	Alumni configuration assertions passed.	**PASS**
TC-AUTH-05	Bcrypt hash validates the correct password and rejects a wrong password.	Both comparison assertions passed.	**PASS**
TC-AUTH-06	Institutional email examples are accepted; public/other domains are rejected by the script's local validation expression.	All five positive/negative examples passed. The script does not invoke the registration API.	**PASS**
TC-AUTH-07	LinkedIn URLs normalize valid forms and reject invalid/javascript schemes.	All five helper assertions passed.	**PASS**
TC-AUTH-08	Public profile sanitization strips phone fields and hides private, unconfirmed LinkedIn data.	All asserted sanitized output fields matched expectations.	**PASS**

4.4 2022 dataset and privacy — `test2022ImportAndPrivacy.js`

ID	Test / expected result	Observed result	Status
TC-DATA-01	Dataset contains 48 records and no phone/secondary-phone fields.	All 48 records satisfied the checked field-exclusion assertions.	**PASS**
TC-DATA-02	Company normalization handles known variants and empty/null values.	All six normalization assertions passed.	**PASS**
TC-DATA-03	Sanitized alumni output enforces LinkedIn visibility for public, student and admin viewers while excluding phone fields.	Public, student, permitted-viewer and admin fixture checks passed.	**PASS**

4.5 Duplicate registration scenarios — `testDuplicateRegistrationFlow.js`

ID	Test / expected result	Observed result	Status
TC-DUP-01	Whitespace-trimmed and mixed-case email values normalize to lowercase canonical values.	All listed normalization examples passed.	**PASS**
TC-DUP-02	Non-institutional and malformed email examples are rejected by the script's local validation expression.	Positive and negative example assertions passed; no API request was made.	**PASS**
TC-DUP-03	Duplicate email scenario yields the expected 409 response shape and message without sensitive fields.	Script-built duplicate response simulation matched expected values; live controller/route was not exercised.	**PASS**
TC-DUP-04	Duplicate attempt must not overwrite existing account fixture data or password.	In-memory fixture assertions passed; no database persistence was exercised.	**PASS**
TC-DUP-05	New-account login scenario accepts the matching bcrypt password and active account fixture.	In-memory registration/login simulation passed; no live registration or login API was exercised.	**PASS**

5. Findings and Defect/Discrepancy Log

The existing typography tests were not modified. Findings below preserve their literal outcomes and distinguish a failed test assertion from a confirmed runtime defect.

ID	Evidence	Finding	Recommended disposition
F-01	TC-TYP-01, TC-UI-05	Both scripts expect a Helvetica font option. The current theme configuration instead exposes Arial, Roboto and Times variants, so both assertions fail.	Confirm the approved font requirement. Then align either the implementation or the two test expectations and rerun those tests.
F-02	TC-TYP-03	The pre-hydration check expects the exact stack `'Times New Roman', Times, serif`; the current stack includes an additional Georgia fallback. This is an exact-stack mismatch, not absence of Times New Roman.	Decide whether the fallback is allowed and make the expected stack consistent in the source and test.
F-03	TC-TYP-08	The test rejects any Roboto reference, while the current theme and CSS intentionally include Roboto. The assertion therefore fails against the current configuration.	Resolve the font policy and update implementation or test rule accordingly; do not mark the assertion passed until rerun.

> **Classification:** The above are verified test failures and source/test expectation discrepancies. The tests alone do not establish whether the current font choices are acceptable product behavior; that decision depends on the approved typography requirement.

6. Quality Summary, Limitations and Next Actions

Measured results

- **Execution rate:** 32 / 32 listed cases executed = 100% for this selected set.
- **Pass rate:** 28 / 32 executed cases passed = 87.5%.
- **Open failed cases:** 4 typography assertions (TC-TYP-01, TC-TYP-03, TC-TYP-08 and TC-UI-05).
- **Not run:** No unexecuted case is counted as a pass. Database/API smoke, SMTP, realtime and other integration scripts are outside this run.

Limitations

- Several suites validate source strings or local helper logic, not the running browser application.
- Duplicate-registration scenarios include in-memory simulations and do not verify a database-backed controller response.
- Responsive breakpoints were checked as class-name presence, not by viewport screenshots or browser automation.
- There is no claim of full requirements coverage, performance certification, or security certification from this limited run.

Recommended follow-up

1. Confirm the approved font set and fallback policy with the project owner.
2. Update only the conflicting typography assertions or implementation after that decision; rerun the two typography scripts.
3. Run the database-backed smoke test only against an explicitly isolated test database after confirming its URI and reseed behavior.
4. For broader assurance, add or run non-destructive API integration tests and real browser checks as separate, clearly identified test activities.

7. Test Execution Procedure and Sign-off

1. Use the project repository and the same dependency/runtime environment used for this execution.
2. Run only the existing scripts listed in Section 3 using Node.js.
3. Compare each assertion with its documented expected result and retain the console output as execution evidence.
4. Resolve F-01 through F-03 based on the agreed typography requirement, then rerun the relevant existing tests.
5. Record any later integration/browser tests separately; do not merge unexecuted cases into the passing count.

Role	Name / signature	Date
Prepared by		
Reviewed by		
Approved by		

End of report.